

menu_cli

▶ View Source

def print_menu():

▶ View Source

Exibe a interface gráfica textual (CLI) com as opções disponíveis.

def main():

▶ View Source

Função principal que gere o loop de eventos e a lógica de seleção.

Implementa um sistema de 'match-case' para despachar as chamadas de funções especializadas de acordo com a entrada do utilizador, garantindo a segregação de responsabilidades entre os módulos.

CheckingFiles.PortScanner

► View Source

def nmap_scan():

► View Source

Executa uma varredura de portas TCP num endereço IP e intervalo definidos.

A função utiliza o método TCP Connect Scan (`-sT`), que completa o handshake de três vias do TCP. Este método é fiável e não requer privilégios de root, embora seja mais facilmente detetável por sistemas de deteção de intrusão (IDS).

Metodologia: 1. Validação de Input: Garante que o IP e o range de portas são válidos. 2. Scanning: O Nmap interage com a pilha de rede para testar as portas. 3. Parsing: Os resultados são extraídos do objeto scanner para exibição.

AttackFiles.dos_attack

► View Source

def dos_attack():

► View Source

Executa a rotina de ataque UDP Flood direcionada a um IP alvo.

A função solicita ao utilizador um endereço IP válido, gera um payload aleatório de grande dimensão (65000 bytes) e inicia um ciclo infinito de envio de pacotes UDP percorrendo todo o range de portas (1-65535).

Mecanismo de Auditoria: - Protocolo: UDP (Connectionless) - Payload: Dados aleatórios gerados via urandom. - Vetor: Exaustão de recursos de rede e CPU.

AttackFiles.syn_flood_attack

► View Source

def syn_attack():

► View Source

Executa a rotina de inundação SYN Flood contra um alvo específico.

A função solicita um IP e uma porta, constrói pacotes TCP customizados com a flag 'S' (SYN) ativada e envia-os em loop. Cada pacote utiliza uma porta de origem aleatória para simular múltiplos clientes distintos.

Mecanismo de Exploração: 1. O atacante envia um pacote SYN (Pedido de conexão). 2. O servidor responde com SYN-ACK e reserva recursos na memória. 3. O atacante ignora o SYN-ACK e não envia o ACK final. 4. O servidor mantém a conexão pendente até atingir o timeout.

Análise técnica: - Camada 3 (IP): Define o endereço de destino (dst). - Camada 4 (TCP): Define portas aleatórias (RandShort), porta alvo e flag SYN (flags="S"). - Payload (Raw): Adiciona 1KB de dados irrelevantes para aumentar o consumo de banda.

API Documentation

GEOIP_DB_PATH

get_ip_info()

format_http_date()

class LogReport

header()

footer()

generate_pdf_report()

generate_csv_report()

analyze_logs()

CheckingFiles.LogReporter

[► View Source](#)

GEOIP_DB_PATH = './CheckingFiles/GeoLite2-City.mmdb'

def get_ip_info(ip):

[► View Source](#)

Consulta a base de dados GeoLite2 para obter a localização geográfica de um IP.

Args: ip (str): Endereço IP extraído dos logs.

Returns: tuple: (País, Cidade) ou informações de erro/desconhecido.

def format_http_date(date_str):

[► View Source](#)

Normaliza a data do formato Apache para o formato ISO.

Args: date_str (str): Data original (ex: 24/Jan/2026:19:17:02 +0000).

Returns: str: Data formatada (YYYY-MM-DD HH:MM:SS).

class LogReport(fpdf.fpdf.FPDF):

[► View Source](#)

Classe personalizada para geração de PDF com cabeçalho e rodapé automáticos.

def header(self):

[► View Source](#)

Header to be implemented in your own inherited class

This is automatically called by `FPDF.add_page()` and should not be called directly by the user application. The default implementation performs nothing: you have to override this method in a subclass to implement your own rendering logic.

Note that header rendering can have an impact on the initial (x,y) position when starting to render the page content.

def footer(self):

[► View Source](#)

Footer to be implemented in your own inherited class.

This is automatically called by `FPDF.add_page()` and `FPDF.output()` and should not be called directly by the user application. The default implementation performs nothing: you have to override this method in a subclass to implement your own rendering logic.

def generate_pdf_report(data_list, filename='Relatorio_Logs.pdf'):

[► View Source](#)

Cria um ficheiro PDF formatado com a tabela de eventos de segurança.

def generate_csv_report(data_list, filename='relatorio_ataques.csv'):

[► View Source](#)

Exporta os dados da análise para o formato CSV.

def analyze_logs():

[► View Source](#)

Executa a rotina principal de análise de logs de sistema e web.

A extração é baseada em padrões de Regex que identificam: - Falhas de autenticação SSH (auth.log). - Códigos de erro HTTP (401, 403, 404) que indicam brute-force ou scanning.

CheckingFiles.PortKnocker

► View Source

def knock():

► View Source

Executa a sequência de 'batidas' (knocks) num IP alvo.

A função interage com o utilizador para definir o alvo e a sequência de portas. Utiliza o Scapy para construir pacotes TCP com a flag SYN personalizada, garantindo que a sequência é enviada com intervalos controlados para evitar problemas de race condition no processamento do servidor.

Análise Técnica: - Protocolo: TCP Camada 4. - Flags: SYN ("S"). - Intervalo: 1 segundo entre pacotes para integridade da sequência.

Search...

API Documentation

- DB_NAME
- PRIVATE_KEY_PATH
- PUBLIC_KEY_PATH
- OTP_SECRET_PATH
- setup()
- load_public_key()
- load_private_key()
- encrypt_data()
- decrypt_data()
- authenticate()
- add_credential()
- list_credentials()
- update_credential()
- delete_credential()
- password_menu()

PasswordFiles.PasswordManager

► View Source

DB_NAME = './PasswordFiles/passwords.db'

PRIVATE_KEY_PATH = './PasswordFiles/private_key.pem'

PUBLIC_KEY_PATH = './PasswordFiles/public_key.pem'

OTP_SECRET_PATH = './PasswordFiles/otp_secret.txt'

def setup(): ► View Source

Inicializa o ambiente de segurança do gestor.

Gera o par de chaves RSA se inexistente, configura o segredo TOTP para o segundo fator de autenticação e cria a estrutura de tabelas no SQLite. O QR Code para configuração da app móvel é gerado automaticamente no primeiro setup.

def load_public_key(): ► View Source

Carrega a chave pública do disco para operações de cifragem.

def load_private_key(): ► View Source

Carrega a chave privada do disco para operações de decifragem.

def encrypt_data(data): ► View Source

Cifra uma string de texto limpo utilizando RSA-OAEP.

Args: data (str): Texto a ser protegido. Returns: bytes: Dados cifrados binários.

def decrypt_data(cipher_text): ► View Source

Decifra dados binários para recuperar o texto original.

Args: cipher_text (bytes): Dados recuperados da base de dados. Returns: str: Texto limpo decifrado.

def authenticate(): ► View Source

Valida a identidade do utilizador através de um token TOTP.

Returns: bool: True se o código estiver correto e dentro da janela temporal.

def add_credential(): ► View Source

Interage com o utilizador para cifrar e guardar uma nova credencial no SQLite.

def list_credentials(): ► View Source

Lista todas as credenciais após validação 2FA bem-sucedida.

def update_credential(): ► View Source

Atualiza campos específicos de um registo após validação 2FA.

def delete_credential(): ► View Source

Remove permanentemente um registo da base de dados após validação 2FA.

def password_menu(): ► View Source

Interface principal do Gestor de Passwords.

API Documentation

- MSG_FOLDER
- CLIENT_KEYS_FOLDER
- PUB_KEY_FILE
- PRIV_KEY_FILE
- decrypt_blob()
- save_message()
- start_server()
- generate_keys()

ChatFiles.server_chat

► View Source

MSG_FOLDER = 'server_messages'

CLIENT_KEYS_FOLDER = 'client_keys'

PUB_KEY_FILE = 'public_key.pem'

PRIV_KEY_FILE = 'private_key.pem'

def decrypt_blob(encrypted_blob):

► View Source

Decifra um blob binário utilizando a chave privada RSA do servidor.

Args: encrypted_blob (bytes): Dados cifrados recebidos via socket.

Returns: bytes: Texto limpo original decifrado ou None em caso de falha.

def save_message(user, encrypted_blob):

► View Source

Guarda uma mensagem cifrada num ficheiro binário de arquivo.

Utiliza um formato de prefixo de 4 bytes (Big Endian) para armazenar o tamanho do blob, permitindo a leitura sequencial de múltiplas mensagens.

Args: user (str): Identificador do utilizador/proprietário do arquivo. encrypted_blob (bytes): O conteúdo cifrado a ser persistido.

def start_server():

► View Source

Inicia o loop principal do servidor TCP.

Gere os seguintes comandos do protocolo: - REQ_PUBKEY: Envia a chave pública do servidor ao cliente. - SEND: Recebe e armazena mensagens, associando a chave pública do cliente. - GET: Realiza a re-criptação do arquivo para a chave do cliente e envia. - DEL: Remove permanentemente mensagens e chaves de um utilizador.

def generate_keys():

► View Source

Gera o par de chaves RSA do servidor caso não existam ficheiros PEM.

Cria uma chave de 2048 bits utilizando o expoente público padrão (65537).

API Documentation

- SERVER_IP
- PORT
- MY_USERNAME
- CLIENT_PRIV_KEY
- CLIENT_PUB_KEY
- generate_client_key()
- get_server_public_key()
- encrypt_message()
- send_secure_msg()
- decrypt_local_archive()
- download_messages()
- export_backup_aes()
- delete_server_data()
- client_chat_menu()

ChatFiles.python-client

[► View Source](#)

SERVER_IP = '192.168.1.86'

PORT = 9999

MY_USERNAME = 'paulo_abade'

CLIENT_PRIV_KEY = 'client_private.pem'

CLIENT_PUB_KEY = 'client_public.pem'

def generate_client_key():

[► View Source](#)

Gera o par de chaves RSA (Pública e Privada) do cliente se não existirem no disco.

A chave privada é guardada sem cifragem (NoEncryption) para facilitar a automação, e a chave pública é exportada no formato SubjectPublicKeyInfo para partilha.

def get_server_public_key():

[► View Source](#)

Realiza o handshake com o servidor para obter a sua chave pública.

Returns: str: Caminho do ficheiro temporário com a chave do servidor ou None em caso de erro.

def encrypt_message(message, pub_key_path):

[► View Source](#)

Cifra uma string utilizando RSA com padding OAEP.

Args: message (str): O texto limpo a cifrar. pub_key_path (str): Caminho para o ficheiro .pem da chave pública de destino.

Returns: bytes: O conteúdo cifrado resultante.

def send_secure_msg():

[► View Source](#)

Fluxo de envio de mensagem segura. Obtém a chave do servidor, cifra a mensagem e envia o pacote contendo o identificador do utilizador e a sua própria chave pública.

def decrypt_local_archive():

[► View Source](#)

Processa e decifra o arquivo binário descarregado do servidor.

Lê o ficheiro binário sequencialmente, extraindo o tamanho de cada blob cifrado e utilizando a chave privada local para recuperar o texto original.

def download_messages():

[► View Source](#)

Descarrega o histórico de mensagens do servidor. O servidor é responsável por decifrar o histórico com a sua chave e re-cifrá-lo com a chave deste cliente.

def export_backup_aes():

[► View Source](#)

Realiza o backup de um texto para um ficheiro encriptado simetricamente (AES-256).

Utiliza PBKDF2 para derivar uma chave segura a partir de uma password definida pelo utilizador, aplicando um salt estático.

def delete_server_data():

[► View Source](#)

Envia um pedido de remoção definitiva de todos os dados do utilizador no servidor. Isto inclui o histórico de mensagens e a chave pública armazenada.

def client_chat_menu():

[► View Source](#)

Loop principal que gere a interface de texto do cliente.